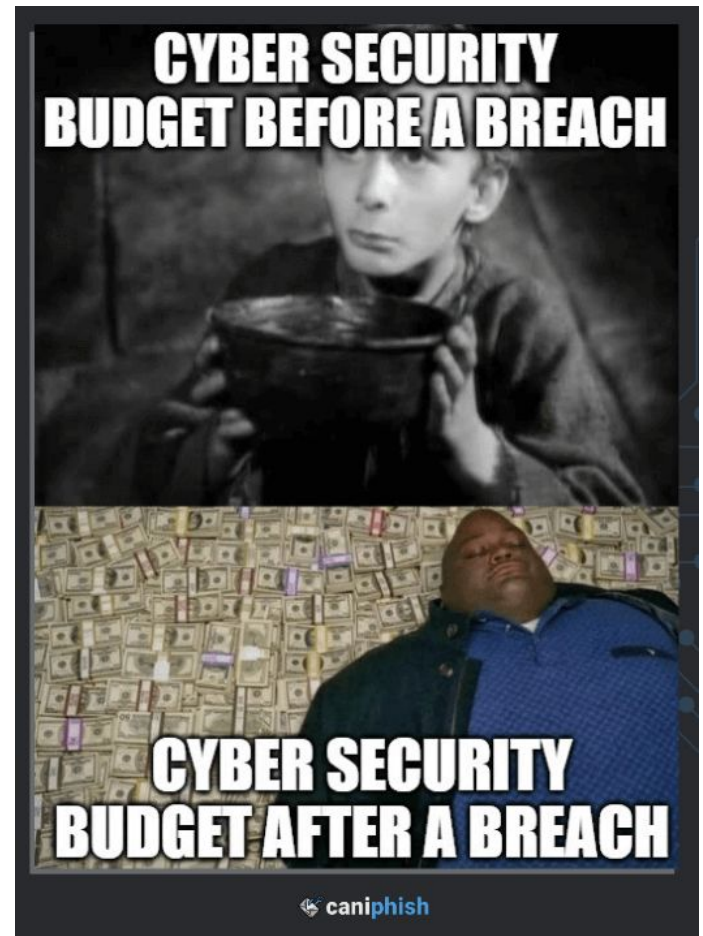


Offense and Defense (Overall Summary)

Kameswari Chebrolu



<https://d3p8e1mvy30w84.cloudfront.net/assets/images/blog/cyber-security-memes/budget-before-after-meme.png>

Outline

- How are web attacks carried out?
- How to defend against them?
 - Web Application Firewalls
- Overall Best Practices to follow (Generic)

Attack Process

How are attacks typically carried out?

- Reconnaissance (Information Gathering): Gather information about the target web application
 - Technologies used (e.g., server software, frameworks, libraries) using tools like Wappalyzer or WhatWeb
 - Cross-reference with CVE databases to find known vulnerabilities (e.g., outdated versions)
 - Use ZAP to map the site structure
 - Identify all endpoints, hidden pages, and forms using spiders and manual exploration

- Scanning and Enumeration: Actively probe the web application to map its structure and identify specific vulnerabilities
 - Automated vulnerability scanners like OWASP ZAP, Burp Suite, Nikto, or Acunetix
 - Examine input fields and parameters for injection points
 - Analyze cookies and session tokens
 - Identify misconfigurations, outdated software, or missing security patches
 - Enumerate user accounts or roles via error messages or predictable pattern

- Gaining Access (Exploitation): Exploit vulnerabilities to gain unauthorized access to the web application
 - Injection Attacks: SQL injection, Command injection, XML external entity (XXE)
 - Cross-Site Scripting (XSS): Injecting malicious scripts to run in users' browsers
 - Broken Authentication: Exploiting weak login mechanisms, session fixation, or credential stuffing
 - Insecure File Uploads: Uploading malicious files or web shells
 - Path Traversal: Accessing sensitive files on the server
 - Cross-Site Request Forgery (CSRF): Forcing logged-in users to execute malicious actions etc

- Privilege Escalation: Elevate access rights to gain more control over the application or underlying infrastructure
 - Exploiting insecure direct object references (IDOR) to access restricted resources
 - Leveraging insecure role-based access control (RBAC) to access admin-level functionality
 - Hijacking sessions or tokens for privileged accounts
 - Exploiting vulnerabilities in the web server, database, or operating system

- Lateral Movement: Move within the network or exploit other related systems connected to the compromised application
 - Accessing backend databases using SQL injection
 - Interacting with APIs to access additional functionality or data
 - Using the compromised application as a pivot point to attack internal systems or other applications
 - Exploiting trust relationships between systems

- Exfiltration: Steal sensitive data or assets from the application
 - Databases (e.g., extracting user credentials, PII, or financial data)
 - Session cookies or tokens for further impersonation
 - Sensitive files (e.g., configuration files, logs, or source code)
 - API keys and credentials

- Maintaining Access (Persistence): Maintain control over the compromised web application for further attacks or data exfiltration
 - Installing web shells for remote access (e.g., PHP-based shells)
 - Leveraging backdoors in source code or server configurations
 - Adding rogue admin accounts or API keys

- Covering Tracks: Erase evidence of the attack to avoid detection
 - Delete logs or modifying log entries to hide malicious activities
 - Clean up uploaded web shells or malicious files
 - Obfuscate payloads to make analysis harder
 - Use encrypted or encoded communication for data exfiltration.

Defense

- Reconnaissance:
 - Limit public information exposure
 - Use tools like WAFs (Web Application Firewalls) to detect and block reconnaissance activities
- Scanning and Enumeration:
 - Monitor unusual scanning activity in server logs
 - Harden application endpoints and implement rate limiting

- Exploitation:
 - Regularly patch software and frameworks
 - Perform security assessments (e.g., penetration testing, SAST, DAST) and fix issues
 - Enforce best security coding practices ; data validation
- Privilege Escalation:
 - Implement least-privilege access controls
 - Use role-based access control (RBAC) with proper testing
- Lateral Movement:
 - Segment the network to limit attack propagation
 - Restrict access to internal systems via firewalls or proxies and also via authentication

- Exfiltration:
 - Encrypt sensitive data in transit and at rest
 - Monitor unusual outbound traffic patterns
- Persistence:
 - Regularly audit application configurations and accounts.
 - Detect and remove rogue files or accounts
- Covering Tracks:
 - Use logging and monitoring solutions with tamper-proof mechanisms
 - Employ forensic tools to analyze suspicious activity

Web Application Firewalls

- Firewall that protects web applications by filtering and monitoring **HTTP traffic** from Internet
 - Operates at the application layer
 - Inspects both incoming and outgoing traffic
- Can identify and block a variety of attacks
 - SQL injection, cross-site scripting (XSS), cross-site request forgery (CSRF), DDOS etc
- Can help organizations meet regulatory compliance requirements
 - PCI DSS, HIPAA, and GDPR etc

Types of WAFs

- Hardware-Based: Physical devices installed at the network perimeter
 - High performance and low latency but costly
 - Complex deployment and configuration
 - Suitable for large enterprises, data centers, regulated industries!
 - Imperva SecureSphere, Barracuda Web Application Firewall, Citrix Netscaler Application Firewall etc

- Network-Based: Hardware or virtual appliance installed at network edge
 - Similar to hardware WAF if not virtual
 - "Hardware WAF" refers to the physical appliance form factor
 - "Network-based WAF" focuses on its placement within the network architecture
 - Suitable for handling high volumes of traffic
 - Requires specialized network knowledge for management

- Host-Based: Software modules or agents installed on the web server
 - Deep integration with web application allows high customization
 - Can Consume server resources, affecting performance
 - Requires management and maintenance on each server
 - E.g. Ngnix ModSecurity, Owasp Coraza (open source)

- Cloud-Based: Delivered as a cloud service by third-party providers
 - Not your headache, infrastructure wise!
 - Incoming traffic destined for on-premises web servers is routed through cloud-based WAF
 - Achieved through DNS changes or a reverse proxy
 - Easily scales to handle large traffic volumes
 - Potential additional latency depending on provider proximity
 - E.g. AWS WAF, Cloudflare WAF, Akamai Kona Site Defender

- Hybrid WAF: Combination of network-based, host-based, and cloud-based WAFs
 - Different parts of the application infrastructure protected differently
 - Increased complexity in configuration and management

Models

Based on approach to traffic filtering and rule enforcement

- Positive Security Model
 - "Allowlist": defines what is allowed and blocks everything else by default
 - Only known, trusted, and explicitly defined inputs are permitted
 - Provides higher security assurance but high maintenance overhead
 - Requires continuous updating of allowlists
 - Potential for Blocking Legitimate Traffic
 - Works in environments where security is prioritized over convenience :-)

- Negative Security Model
 - "Blocklist": defines what is not allowed, permitting all other traffic by default
 - Relies on patterns, signatures, and rules to identify and block known malicious traffic
 - Easier to deploy initially but has false negatives
 - May miss new or unknown threats that aren't covered by the existing rules and signatures
 - Reactive Approach: Requires constant updating to keep up with emerging threats
 - Works in organizations needing a more flexible and adaptable security solution

- Mixed Security Model: Combines elements of both positive and negative security models
 - Uses allowlists for known, critical inputs
 - Uses blocklists to filter out known malicious patterns
 - A balanced approach that leverages the strengths of both models
 - More complex to configure and manage due to the dual nature of the rules
 - Complex applications where both known good inputs and dynamic traffic need to be managed

Modes

Based on how WAF interacts with traffic

- Passive Mode: WAF monitors and analyzes traffic without actively blocking or modifying it
 - Generates alerts for suspicious or malicious activities but allows all traffic to pass through
 - Zero Latency since it doesn't alter traffic
 - Useful during initial deployment phase to understand the traffic patterns
 - Can fine-tune the rules before moving to an active mode
 - Drawbacks:
 - Does not block malicious traffic, so application remains vulnerable
 - Requiring manual intervention to respond to threats

- Active Mode: Actively filters, blocks, and modifies traffic based on predefined rules and policies
 - Can prevent malicious traffic in real-time
 - Risk of False Positives: May block legitimate traffic if the rules are too strict or not properly tuned
 - Can introduce some latency due to the inspection and filtering processes
 - Requires careful configuration and continuous tuning to balance security and accessibility

Deployment Models

- Reverse Proxy Deployment: WAF acts as an intermediary between clients and web server
 - Breaks the connection in between client server
 - HTTPS → WAF needs access to private keys
 - Requires changes in DNS settings or configurations to direct traffic through the WAF
 - Adds an extra hop in the traffic flow, which can introduce latency
 - Single Point of Failure: If WAF goes down, can disrupt access to the web application

- Inline Deployment: WAF placed directly in the path of traffic between the client and the web server
 - Can break connection too!
 - Any issues with the WAF can cause downtime for web application

- Out-of-Band Deployment (Passive/Monitor Mode): WAF monitors a copy of the traffic through a network tap or port mirroring
 - HTTPS traffic → SSL offloading has to happen in some front-end device
 - No Latency
 - No Risk of Disruption
 - No Real-Time Blocking
 - Can only alert on malicious traffic, not block it in real-time
 - Requires manual intervention to mitigate threats

- Transparent Bridge Mode: Operates as a bridge at the network layer (Layer 2)
 - Rather than as a traditional proxy at the application layer (Layer 7)
 - Useful in scenarios where existing network setups cannot accommodate IP or DNS changes
 - Complex Configuration: Can be more complex to set up and maintain
 - Potentially Limited Features: Some advanced features may not be available in transparent mode

Internal Mechanism

- Signature-Based Detection:
 - Employ predefined signatures or patterns of known attacks
 - When incoming traffic matches a known attack signature, block or log the request
 - For XSS attacks, sample signature in URL
 - `<\s*script[^>]*>(.*?)<\s*/\s*script\s*>` (Matches any `<script>` tag with any content inside it and closes with a `</script>` tag.)
 - For SQL injection, sample signature in URLs
 - `'(\s|;|--)|(\s|;|--)'[^']` (use of the single quote character in unexpected places)

- Anomaly-Based Detection:
 - Establish a baseline of normal behavior for web traffic
 - Request rates, HTTP methods, sizes of payloads, URL patterns
 - Flag requests or responses that deviate significantly from this baseline
 - Can help detect new or zero-day attacks that may not have a known signature

- IP Reputation and **Rate Limiting**
 - WAFs may maintain lists of known malicious IP addresses (IP reputation lists)
 - Use rate limiting to protect against brute force attacks, DDoS
- Integration with external Threat Intelligence
 - Dynamically update detection rules and signatures based on emerging threats

- Logging and Reporting
 - Log all detected events, including blocked requests, anomalies, and other security incidents
 - Useful for forensic analysis, compliance auditing, and improving effectiveness of policies
- Response Actions
 - block the request
 - redirect to a CAPTCHA challenge page
 - alerting administrators, or modifying the request or response to mitigate the threat

Rate Limiting

- Count Requests: Tracks the number of requests in a specific time frame
 - Particular client (often identified by IP address or API key)
 - Type of request
- Thresholds: Maximum number of requests allowed in a given period
- Blocking or Throttling: Exceed the rate limit → block or slow down the response rate (throttling)
- Response Codes: Clients informed of rate limiting via HTTP status codes such as 429 (Too Many Requests).

- Balancing Limits:
 - Setting limits too low can disrupt legitimate users
 - Setting them too high might not effectively prevent attacks
- Distributed Attacks:
 - Rate limiting per client not very effective against distributed attacks (DDoS)

Limitations

- Cost: Hardware WAFs and advanced cloud-based solutions can be expensive
- Complexity
 - Configuration and Tuning requires expertise to avoid false positives and false negatives
 - False Positives: Legitimate traffic may be blocked if the WAF rules are too strict
 - False Negatives: Malicious traffic may go undetected if WAF rules are too lenient or not updated

- Performance Impact
 - Inspecting and filtering traffic can introduce latency, potentially affecting application performance
 - Can consume significant network and server resources, particularly in high-traffic environments
- Effectiveness of a WAF heavily depends on its configuration
 - Poorly configured WAFs can provide a false sense of security!



https://media.licdn.com/dms/image/v2/D5622AQF5JbsQLHGOVg/feedshare-shrink_800/feedshare-shrink_800/0/1714572312149?e=2147483647&v=beta&t=U16TlbM0AyDrbLYsBmZlqDkK6s3AXJJPRnVe7Ek-i8



Best Practices

Security is Economics

Adi Shamir: “ There are no secure systems, only degrees of insecurity”

- No system is 100% secure
- Tradeoff between cost and security
 - \$10,000 WAF to protect \$1000 worth of secrets?
- Focus on securing the weakest link
 - Screen door and main door locked with expensive deadbolt

Least Privilege

- Give only minimum required privileges to get job done
 - Web applications can only read or write to specific directories/tables necessary for their operation
 - Limit API keys and tokens to specific actions and scopes required by the application
 - Implement roles with specific permissions tailored to the needs of different types of users.
 - Configure firewalls to allow only necessary traffic to and from the web server

Use fail-safe defaults

- Access to resources should be denied by default
- Permissions should be explicitly granted based on the principle of least privilege
 - Configure firewalls to block all traffic by default and only allow required traffic through rules
 - Default passwords should be strong and unique
 - Default settings should not expose any unnecessary services or data
 - Access to sensitive areas of the application requires authentication by default
 - Implement strict CSPs that disallow potentially dangerous actions by default,
 - Use whitelisting (allowing only known good inputs) rather than blacklisting (blocking known bad inputs) for form fields and user inputs

Separation of responsibility

- Split up responsibility, no one person has complete power
 - Nuclear missile launch has two launch officers
 - Purchase committee to evaluate purchases
- Separate roles for developers, testers, and administrators;
Separate roles for detecting, investigating, and resolving security incidents
 - Each group has access only to the resources necessary for their tasks
- Independent auditors to review security practices and compliance regularly

Defense in depth

- Use multiple redundant protections
 - Supandi Joke
- All need to be breached before system is endangered
- Web Applications
 - Implement secure coding standards and practices to minimize vulnerabilities during development lifecycle
 - Deploy WAFs to filter and monitor HTTP traffic between a web application and Internet
 - Ensure all user inputs are validated and sanitized
 - Use Secure server configuration (CSP, .htaccess etc)
 - Implement MFA to require multiple forms of verification

Psychological acceptability/Human Oriented Design

- Users need to buy into the security model
 - E.g. firewall blocking leads to tunnels; Passwords strength leads to post-its
- Implement single sign-on (SSO) solutions to allow users to log in once and gain access to multiple systems
- Make it easy for users to change their passwords or understand the status of their security settings

- If a password is rejected, explain why and offer suggestions for creating a stronger password
- Reduce authentication requirements when users log in from familiar devices or locations but increase them when anomalies are detected!
- Automate software updates and security patches to ensure systems are up to date

- Design should be easy to use for ordinary people
 - NSA's encryption key example
 - Key is stored in a physical token which looks like a door key
 - Very intuitive even for people with no training in crypto
- Use visual cues like color coding (green for safe, red for danger) to communicate security status
- If a user encounters a security warning, offer a brief explanation and clear steps to resolve it
- Provide clear and easy-to-use channels for reporting security incidents or suspicious activities

Ensure Complete Mediation

- Every access to a system's resources is checked for appropriate permissions and access rights
 - Implement centralized mechanisms for access control where permissions and policies are enforced consistently
 - Require users to re-enter their passwords when accessing high-risk areas of the application
 - Enforce access controls on files, databases, and other resources at every access attempt
 - Validate all input parameters every time they are received
 - Log all access attempts and security checks to maintain an audit trail

Detect if you can't prevent

- Detect; if possible provide a way to recover and identify the attacker
 - Example in hardware setting
 - Type 3 Tamper-resistance: make intrusion difficult, usually by employing hardened casing
 - Type 2 Tamper-evident: make intrusion attempts evident to subsequent viewers
 - Often by employing seals which must be broken during intrusion
 - Type 1 Tamper-responsive: detect intrusion attempt and destroy the contents
 - Type 1 most expensive, type 3 more expensive than type 2

- Implement continuous monitoring of networks, systems, and applications to detect anomalies and potential security incidents in real-time
- Incorporate threat intelligence feeds to stay updated on the latest threats and attack techniques

Don't rely on security through obscurity

- Keeping design, algorithm, code, URLs secret is brittle
- Don't trust such claims

Design security in from start

- Fitting in security later is a very tough task (as seen in Internet)
- Backward compatibility will make things insecure
- Conservative Design: Design against the worst security failure under assumptions favourable to the attacker
 - Kerkhoff's principle: Attacker knows all internal details
- Proactive study of attacks: Attempt to break your own system before others do it

Know your threat model

- Assumptions originally made may not be valid
 - E.g. design of Internet
- Identify Assets:
 - Types of data your web application handles (personal, IP or credentials etc)
 - Determine critical components of your system (databases, servers, APIs, and third-party integrations)
- Determine Potential Adversaries:
 - External Attackers, Insiders, Contractors, Competitors

- Identify Threats and Attack Vectors:
 - Common Attacks: SQLi, XSS, CSRF, DDos
 - Advanced Persistent Threats (APTs): more sophisticated threats that may involve long-term, targeted attacks aimed at specific high-value assets
 - Supply Chain Attacks: risks associated with third-party components and services
- Existing Controls: Evaluate the security controls currently in place
 - Firewalls, encryption, authentication mechanisms, secure server configuration etc
- Gaps and Weaknesses: Identify any gaps or weaknesses in existing security measures

- **Assess Likelihood and Impact:**
 - Estimate likelihood of each identified threat occurring based on factors like past incidents, industry trends, and your specific environment
 - Impact: Evaluate the potential impact of each threat on your organization
 - Data loss, financial damage, reputational harm, and legal consequences.
- **Risk Assessment:** Perform a risk assessment to prioritize threats based on their likelihood and potential impact
- **Resource Allocation:** Allocate resources and efforts proportionally to the severity and priority of the threats identified

- Develop Mitigation Strategies:
 - Preventive Measures: Implement measures to prevent threats
 - Secure coding practices, regular security training, and robust access controls
 - Detective Measures: detect threats through continuous monitoring, logging, and intrusion detection systems
 - Responsive Measures: Prepare for incident response with well-defined procedures, backup strategies, and disaster recovery plans

- Iterate and Update:
 - Regular Review: Regularly review and update your threat model
 - Testing and Validation: Continuously test and validate your security measures
 - penetration testing, and red teaming exercises
- Communication and Awareness:
 - Involve key stakeholders, including management, IT staff, and end-users
- Documentation: Document your threat model

Summary

- Web Application attack process involves a bunch of steps
- Covered defense in each step and also WAFs
 - WAFs: many types, modes, deployment models possible
 - Protect against many attacks
 - Based on combination of signature and anomaly detection
- Overall best practices (adapted to web applications)

Reference

- Generic kill chain:
<https://www.geeksforgeeks.org/cyber-kill-chain/>
- WAFs:
<https://www.vmware.com/topics/web-application-n-firewall>
- Best Practices:
<http://inst.eecs.berkeley.edu/~cs161/sp16/notes/2.1.principles.pdf>